R0bot!c$@Rochamb3au-2026

Botball Online simulator: https://botballacademy.org/
KIPR Wombat (controller) documentation: https://www.kipr.org/doc/index.html
Components Sheet link: Botball Inventory

Composants List	Build List	Parcour List
2 Batteries	1 Bot ball lego bag	1 Cotton ball fuzz
2 battery chargers	4 Mecanum wheels	2 foam block bags
2 controls	1 box metal components	1 bag traffic cones
4 wheels	1 bag wood pallets	1 bot ball plushie
1 Caméra		
2 Capteurs infrarouge		
4		

Table to build

-

Wombat wifi 5776
Password: c11f8e00
IP Address: 192.168.125.1
IP Address (main bot/ Rochambeau): 172.16.14.161:8888
172.16.14.171:8888
KISS IDE : `192.168.125.1:8888`

team home base:
https://www.kipr.org/botball/team-resources/botball-team-homebase

basic robot
wifi 5775-wombat
password: 6a6a4600
IP Address: 192.168.125.1
KISS IDE : `192.168.125.1:8888`

left line sensor = 0

right line sensor = 1

Documentation Code:

1 wheel rotation Counter: 1870
wheels diameters 3 inches
Wheels perimeter: 3π

Moteur avant-droit: moteur 0
Moteur avant-gauche: moteur 1
Moteur arrière-droit: moteur 2
Moteur arrière-gauche: moteur 3

Servo bras: servo 1
Servo pince: 2

Servo pince droit: servo 2
resting position:2000
closed position:1200
Servo pince gauche: servo 3
resting position: 1100
close position:1600

Attraper un petit cube: 500

Code light sensor:

```
int light = analog(3);
while(1){
        if (light < 770){
                code
        }
}
```

C/C++

```
/*****************************/
/* KIPR Wombat Botball Code  */
/*****************************/

#include <kipr/wombat.h>

// Motor mapping
#define FR 0
#define FL 1
#define BR 2
#define BL 3

// Servo mapping
#define SERVO_BRAS 1
#define SERVO_PINCE_DROITE 2
#define SERVO_PINCE_GAUCHE 3

// Arm positions (inverted for your hardware)
const int BRAS_UP = 800;
const int BRAS_DOWN = 1750;
const int BRAS_MIN_SAFE = 800;

const int PINCE_DROITE_REST = 2000;
const int PINCE_DROITE_CLOSED = 1100;

const int PINCE_GAUCHE_REST = 800;
const int PINCE_GAUCHE_CLOSED = 2000;

// IR distances
int ir_value_for_20cm = 1700;
int ir_value_for_1cm = 2800;

void drive(int power) {
    motor(FR, power);
    motor(FL, power);
    motor(BR, power);
    motor(BL, power);
}

void stop_drive() {
    drive(0);
}

void set_arm_position(int pos) {
```

```
    if (pos < BRAS_MIN_SAFE) pos = BRAS_MIN_SAFE;
    set_servo_position(SERVO_BRAS, pos);
}

int main() {

    enable_servos();

    // Start posture
    set_arm_position(BRAS_UP);
    set_servo_position(SERVO_PINCE_DROITE, PINCE_DROITE_REST);
    set_servo_position(SERVO_PINCE_GAUCHE, PINCE_GAUCHE_REST);
    msleep(1000);

    // ------------------------------
    //     *** RED COLOR CHECK ***
    // ------------------------------
    camera_open();
    msleep(1000);

    printf("Searching for RED object...\n");

    int sees_red = 0;

    while (!sees_red) {
        camera_update();

        // Use channel 0 = find_red.conf
        int red_count = get_object_count(0);

        if (red_count > 0) {
            printf("Red object detected!\n");
            sees_red = 1;
        } else {
            printf("No red detected... scanning...\n");
            drive(30); // slowly move forward while searching
            msleep(100);
            stop_drive();
        }
    }

    stop_drive();

    // ------------------------------
    //   Continue with pickup sequence
```

```
// ------------------------------

// Move until 20cm distance
while (analog(0) < ir_value_for_20cm) {
    drive(50);
}
stop_drive();

// Lower arm
set_arm_position(BRAS_DOWN);
msleep(600);

// Close pincers
set_servo_position(SERVO_PINCE_DROITE, PINCE_DROITE_CLOSED);
set_servo_position(SERVO_PINCE_GAUCHE, PINCE_GAUCHE_CLOSED);
msleep(600);

// Raise arm
set_arm_position(BRAS_UP);
msleep(600);

// Turn 90B0
motor(FR, 50);
motor(BR, 50);
motor(FL, -50);
motor(BL, -50);
msleep(2800);
stop_drive();

// Approach wall
while (analog(0) < ir_value_for_1cm) {
    drive(40);
}
stop_drive();

// Drop object
set_arm_position(BRAS_DOWN);
msleep(600);

set_servo_position(SERVO_PINCE_DROITE, PINCE_DROITE_REST);
set_servo_position(SERVO_PINCE_GAUCHE, PINCE_GAUCHE_REST);
msleep(500);

// Arm up again
set_arm_position(BRAS_UP);
```

```
    msleep(600);

    camera_close();
    disable_servos();
    return 0;
}
```

Comments:

- Calibrate the black and white sensors!!! I haven't tested this with the real robot because i cant plug in my mac to the bot

```C/C++
#include <kipr/wombat.h>
#include <stdio.h>

#define LEFT_SENSOR_PORT  1
#define RIGHT_SENSOR_PORT 0

#define BLACK_THRESHOLD 2700

#define LEFT_MOTOR  0
#define RIGHT_MOTOR 3
#define DRIVE_SPEED 50


int main() {

    // Start motors forward
    motor(LEFT_MOTOR, DRIVE_SPEED);
    motor(RIGHT_MOTOR, DRIVE_SPEED);

    while (1) {
        int left_val  = analog10(LEFT_SENSOR_PORT);
        int right_val = analog10(RIGHT_SENSOR_PORT);

        printf("L = %d   R = %d\n", left_val, right_val);

        // Detect black value LOWER than threshold
        int left_on_black  = (left_val  < BLACK_THRESHOLD);
        int right_on_black = (right_val < BLACK_THRESHOLD);

        // If either sensor hits black --> STOP
        if (left_on_black || right_on_black) {
            ao();
            printf("Black line detected! Stopping.\n");
            break;
        }

        msleep(20);
    }

    return 0;
}
```

C/C++

```
void turnRight(int angle, int speed){
  //Pivoter à droite d'un angle donné
  cmpc(0);
  int cible = angle *13;

  while(gmpc(0)>-cible){
    motor(0,-speed);
    motor(3,speed);
  }
  ao();
  msleep(500);
}

void turnLeft(int angle, int speed){
    cmpc(0);
    int cible = angle * 13;

    while(gmpc(0) < target){
        motor(0, speed);
        motor(3, -speed);
    }

    ao();
    msleep(300);
}


void forwardDist(int d, int speed){
  //Avancer d'une distance en cm
  int cible=(d*3600/20.4);
  cmpc(0);

  while(gmpc(0)<cible){
    motor(0,speed);
    motor(3,speed);
  }
  ao();
  msleep(250);
}

int main()
{
  while(analog(2)>730){
    //Avancer jusqu'au noir
```

```
    motor(0,50);
    motor(3,50);
  }
  ao();
  msleep(1000);
  forwardDist(15,40);
  msleep(1000);
  turnRight(90,30);
  ao();
  return 0;
}
```

C/C++

```
#include <stdio.h>
#include <kipr/wombat.h>


const int arm_up = 860;       //servo 1
const int arm_down = 1700;

const int dpince_open = 2000;  //servo 2
const int dpince_close = 1200;

const int gpince_open = 1100;  //servo 3
const int gpince_close = 1800;


void forward(int d) {                                  //Avancer pour une certaine
distance en inch
    int target = (1870*d)/9.42;
    cmpc(0);
    while (gmpc(0) < target){
        motor(0,50);
        motor(3,50);
        motor(1,50);
        motor(2,50);
    }

    ao();
}

void turn_around() {                 // Tourner 180°
    int target = 2400;               // adjust for your robot
    cmpc(0);

    while (gmpc(0) < target) {
        motor(0, 50);    // left side forward
        motor(2, 50);

        motor(1, -50);   // right side backward
        motor(3, -50);
    }

    ao();
}
```

```
void backward(int d) {                                  //Avancer pour une certaine
distance en inch
    int target = (1870*d)/9.42;
    cmpc(0);
    while (abs(gmpc(0)) < target){
        motor(0,-50);
        motor(3,-50);
        motor(1,-50);
        motor(2,-50);
    }

    ao();
}

void left(int d) {                                  //gauche pour une certaine distance
en inch
    int target = (1870*d)/9.42;
    cmpc(0);
    while (gmpc(0) < target){
        motor(0,50);
        motor(1,-50);
        motor(2,-50);
        motor(3,50);
    }

    ao();
}

void right(int d) {                                 //gauche pour une certaine distance
en inch
    int target = (1870*d)/9.42;
    cmpc(1);
    while (gmpc(1) < target){
        motor(0,-50);
        motor(1,50);
        motor(2,50);
        motor(3,-50);
    }

    ao();
}

void forward_line() {                                   //Avancer jusqu'a une ligne
noire
    while (1) {
```

```
        int light_value_1 = analog(1);
        int light_value_2 = analog(2);

        if (light_value_1 >= 1200 && light_value_2 >= 1200) {
                break;
        }

        motor(0,50);
        motor(1,50);
        motor(2,50);
        motor(3,50);
        }
        ao();

}

void grab_Bigcube(){                    //Grabs an object on the platform

    set_servo_position(2, dpince_open);
    set_servo_position(3, gpince_open);
    set_servo_position(1, arm_up);
    msleep(500);
    forward(3);
    set_servo_position(2, dpince_close);
    set_servo_position(3, gpince_close);

}

int main (){
    enable_servos();
    set_servo_position(1, arm_up);
    msleep(500);
    forward(10);
    forward_line();
    left(5);
    grab_Bigcube()

    right(5);



    return 0;
}
```

C/C++

```
#include <stdio.h>
#include <kipr/wombat.h>


const int arm_up = 860;       //servo 1
const int arm_down = 1700;

const int dpince_open = 2000;  //servo 2
const int dpince_close = 1200;

const int gpince_open = 1100;  //servo 3
const int gpince_close = 1800;


void forward(int d) {                                  //Avancer pour une certaine
distance en inch
    int target = (1870*d)/9.42;
    cmpc(0);
    while (gmpc(0) < target){
        motor(0,50);
        motor(3,50);
        motor(1,50);
        motor(2,50);
    }

    ao();
}

void turn_around() {                 // Tourner 180°
    int target = 2400;               // adjust for your robot
    cmpc(0);

    while (gmpc(0) < target) {
        motor(0, 50);    // left side forward
        motor(2, 50);

        motor(1, -50);   // right side backward
        motor(3, -50);
    }

    ao();
}
```

```
void backward(int d) {                                  //Avancer pour une certaine
distance en inch
    int target = (1870*d)/9.42;
    cmpc(0);
    while (abs(gmpc(0)) < target){
        motor(0,-50);
        motor(3,-50);
        motor(1,-50);
        motor(2,-50);
    }

    ao();
}

void left(int d) {                                  //gauche pour une certaine distance
en inch
    int target = (1870*d)/9.42;
    cmpc(0);
    while (gmpc(0) < target){
        motor(0,50);
        motor(1,-50);
        motor(2,-50);
        motor(3,50);
    }

    ao();
}

void right(int d) {                                 //gauche pour une certaine distance
en inch
    int target = (1870*d)/9.42;
    cmpc(1);
    while (gmpc(1) < target){
        motor(0,-50);
        motor(1,50);
        motor(2,50);
        motor(3,-50);
    }

    ao();
}

void forward_line() {                                   //Avancer jusqu'a une ligne
noire
    while (1) {
```

```
       int light_value_1 = analog(1);
       int light_value_2 = analog(2);

       if (light_value_1 >= 1200 && light_value_2 >= 1200) {
              break;
       }

       motor(0,50);
       motor(1,50);
       motor(2,50);
       motor(3,50);
       }
       ao();

}

void grab_Bigcube(){                    //Grabs an object on the platform

    set_servo_position(2, dpince_open);
    set_servo_position(3, gpince_open);
    set_servo_position(1, arm_up);
    msleep(500);
    forward(3);
    set_servo_position(2, dpince_close);
    set_servo_position(3, gpince_close);

}

int main (){
    enable_servos();
    set_servo_position(1, arm_up);
    msleep(500);
    forward_line();
    turn_around();
    right(2);
    grab_smallcube();




    return 0;
}
```

```
#include <stdio.h>
#include <kipr/wombat.h>


const int arm_up = 860;            //servo 1
const int arm_down = 1800;
const int arm_half = 1500;


const int dpince_open = 2000;  //servo 2
const int dpince_close = 1200;

const int gpince_open = 1100;  //servo 3
const int gpince_close = 1800;



void forward(int d) {                                    //Avancer pour une certaine distance en
inch
   int target = (1870*d)/9.42;
   cmpc(0);
   while (gmpc(0) < target){
      motor(0,50);
      motor(3,50);
      motor(1,50);
      motor(2,50);
   }

   ao();
}

void turn_around() {               // Tourner 180B0
   int target = 4200;              // adjust for your robot
   cmpc(0);

   while (gmpc(0) < target) {
      motor(0, 50);    // left side forward
      motor(2, 50);

      motor(1, -50);   // right side backward
      motor(3, -50);
   }

   ao();
```

```
}


void backward(int d) {                          //Avancer pour une certaine distance en
inch
   int target = -1*(1870*d)/9.42;
   cmpc(0);
   while (gmpc(0) > target){
      motor(0,-50);
      motor(3,-50);
      motor(1,-50);
      motor(2,-50);
   }

   ao();
}


void left(int d) {                              //gauche pour une certaine distance en inch
   int target = (1870*d)/9.42;
   cmpc(0);
   while (gmpc(0) < target){
      motor(0,50);
      motor(1,-50);
      motor(2,-50);
      motor(3,50);
   }

   ao();
}

void right(int d) {                             //gauche pour une certaine distance en inch
   int target = (1870*d)/9.42;
   cmpc(1);
   while (gmpc(1) < target){
      motor(0,-50);
      motor(1,50);
      motor(2,50);
      motor(3,-50);
   }

   ao();
}
```

```
void forward_line() {                                   //Avancer jusqu'a une ligne noire
   while (1) {
         int light_value_1 = analog(1);
         int light_value_2 = analog(2);

         if (light_value_1 >= 1200 && light_value_2 >= 1200) {
         break;
         }

         motor(0,50);
         motor(1,50);
         motor(2,50);
         motor(3,50);
         }
         ao();

}

void grab_Bigcube(){                    //Grabs an object on the platform

   set_servo_position(2, dpince_open);
   set_servo_position(3, gpince_open);
   set_servo_position(1, arm_down);
   msleep(500);
   forward(3);
   set_servo_position(2, dpince_close);
   set_servo_position(3, gpince_close);

}

void grab_smallcube(){                          //Grabs an object on the platform

   set_servo_position(2, dpince_open);
   set_servo_position(3, gpince_open);
   set_servo_position(1, arm_down);
   msleep(1000);
   set_servo_position(2, dpince_close);
   set_servo_position(3, gpince_close);

}



int main(){
```

```
    enable_servos();
    set_servo_position(1, arm_up);
    forward_line();
    right(3);
    grab_smallcube();
    msleep(1000);
    set_servo_position(1, arm_up);
    backward(2);
    left(3);
    turn_around();
    forward(3);
    set_servo_position(1, arm_down);
    msleep(500);
    set_servo_position(2, dpince_open);
    set_servo_position(3, gpince_open);
    backward(2);



    return 0;
}
```

C/C++

```
#include <stdio.h>
#include <kipr/wombat.h>


const int arm_up = 860;       //servo 1
const int arm_down = 1800;
const int arm_half = 1500;


const int dpince_open = 2000;  //servo 2
const int dpince_close = 1200;

const int gpince_open = 1100;  //servo 3
const int gpince_close = 1800;



void forward(int d) {                                  //Avancer pour une certaine
distance en inch
    int target = (1870*d)/9.42;
    cmpc(0);
    while (gmpc(0) < target){
        motor(0,50);
        motor(3,50);
        motor(1,50);
        motor(2,50);
    }

    ao();
}

void turn_around() {                 // Tourner 180B0
    int target = 4200;               // adjust for your robot
    cmpc(0);

    while (gmpc(0) < target) {
        motor(0, 50);    // left side forward
        motor(2, 50);

        motor(1, -50);   // right side backward
        motor(3, -50);
    }

    ao();
```

```
}


void backward(int d) {                                  //Avancer pour une certaine
distance en inch
    int target = -1*(1870*d)/9.42;
    cmpc(0);
    while (gmpc(0) > target){
        motor(0,-50);
        motor(3,-50);
        motor(1,-50);
        motor(2,-50);
    }

    ao();
}


void left(int d) {                              //gauche pour une certaine distance
en inch
    int target = (1870*d)/9.42;
    cmpc(0);
    while (gmpc(0) < target){
        motor(0,50);
        motor(1,-50);
        motor(2,-50);
        motor(3,50);
    }

    ao();
}

void right(int d) {                             //gauche pour une certaine distance
en inch
    int target = (1870*d)/9.42;
    cmpc(1);
    while (gmpc(1) < target){
        motor(0,-50);
        motor(1,50);
        motor(2,50);
        motor(3,-50);
    }

    ao();
}
```

```
void forward_line() {                                //Avancer jusqu'a une ligne
noire
    while (1) {
       int light_value_1 = analog(1);
       int light_value_2 = analog(2);

       if (light_value_1 >= 1200 && light_value_2 >= 1200) {
             break;
       }

       motor(0,50);
       motor(1,50);
       motor(2,50);
       motor(3,50);
       }
       ao();

}

void grab_Bigcube(){                    //Grabs an object on the platform

    set_servo_position(2, dpince_open);
    set_servo_position(3, gpince_open);
    set_servo_position(1, arm_down);
    msleep(500);
    forward(3);
    set_servo_position(2, dpince_close);
    set_servo_position(3, gpince_close);

}

void grab_smallcube(){                  //Grabs an object on the platform

    set_servo_position(2, dpince_open);
    set_servo_position(3, gpince_open);
    set_servo_position(1, arm_down);
    msleep(1000);
    set_servo_position(2, dpince_close);
    set_servo_position(3, gpince_close);

}

void grab_pompom(){              //Grabs a pompom from the floor
```

```
    set_servo_position(1, arm_down);
    msleep(1000);
    set_servo_position(2, dpince_open);
    set_servo_position(3, gpince_open);
    msleep(1000);


    int right = dpince_open;
    int left  = gpince_open;

    // Alternating incremental closing
    while (right > dpince_close || left < gpince_close) {

        // CLOSE RIGHT CLAW a little
        if (right > dpince_close) {
            right -= 15;                          // bring inward
            if (right < dpince_close) right = dpince_close;

            set_servo_position(2, right);         // move right claw
            msleep(50);
        }

        // CLOSE LEFT CLAW a little
        if (left < gpince_close) {
            left += 15;                           // bring inward
            if (left > gpince_close) left = gpince_close;

            set_servo_position(3, left);          // move left claw
            msleep(50);
        }
    }
msleep(1000);
}

int main(){
    enable_servos();
    set_servo_position(1, arm_up);
    forward_line();
    right(3);
    grab_smallcube();
    msleep(1000);
    set_servo_position(1, arm_up);
    backward(2);
    left(3);
    turn_around();
```

```
    forward(3);
    set_servo_position(1, arm_down);
    msleep(500);
    set_servo_position(2, dpince_open);
    set_servo_position(3, gpince_open);
    backward(2);



    return 0;
}



int main ()
{
    enable_servos();

    set_servo_position(1,1700);
    msleep(1000);
    for (int i=0;i<900;i++){
        set_servo_position(1,1700-i);
        msleep(50);
    }
    msleep(1000);
    return 0;
}
```

C/C++

```
#include <stdio.h>
#include <kipr/wombat.h>

const int arm_up = 860;       //servo 1
const int arm_down = 1800;
const int arm_half = 1500;


const int dpince_open = 2000;  //servo 2
const int dpince_close = 1200;

const int gpince_open = 1100;  //servo 3
const int gpince_close = 1800;



void forward(int d) {                                  //Avancer pour une certaine
distance en inch
    int target = (1870*d)/9.42;
    cmpc(0);
    while (gmpc(0) < target){
        motor(0,50);
        motor(3,50);
        motor(1,50);
        motor(2,50);
    }

    ao();
}

void turn_around() {                 // Tourner 180B0
    int target = 4200;               // adjust for your robot
    cmpc(0);

    while (gmpc(0) < target) {
        motor(0, 50);    // left side forward
        motor(2, 50);

        motor(1, -50);   // right side backward
        motor(3, -50);
    }

    ao();
```

```
}


void backward(int d) {                                  //Avancer pour une certaine
distance en inch
    int target = -1*(1870*d)/9.42;
    cmpc(0);
    while (gmpc(0) > target){
        motor(0,-50);
        motor(3,-50);
        motor(1,-50);
        motor(2,-50);
    }

    ao();
}


void left(int d) {                              //gauche pour une certaine distance
en inch
    int target = (1870*d)/9.42;
    cmpc(0);
    while (gmpc(0) < target){
        motor(0,50);
        motor(1,-50);
        motor(2,-50);
        motor(3,50);
    }

    ao();
}

void right(int d) {                             //gauche pour une certaine distance
en inch
    int target = (1870*d)/9.42;
    cmpc(1);
    while (gmpc(1) < target){
        motor(0,-50);
        motor(1,50);
        motor(2,50);
        motor(3,-50);
    }

    ao();
}
```

```
void forward_line() {                                  //Avancer jusqu'a une ligne
noire
    while (1) {
       int light_value_1 = analog(1);
       int light_value_2 = analog(2);

       if (light_value_1 >= 1200 && light_value_2 >= 1200) {
             break;
       }

       motor(0,50);
       motor(1,50);
       motor(2,50);
       motor(3,50);
       }
       ao();

}

void turn_left(int angle, int speed){

  cmpc(2);
  while(gmpc(2) < angle){
       //left backward
    motor(1,-50);
       motor(3,-50);
//right forward
       motor(0,50);
    motor(2,50);
  }
  ao();
  msleep(500);
}

void turn_right(int angle, int speed){

  cmpc(2);
  while(gmpc(2) >- angle){
       //left backward
    motor(1,50);
       motor(3,50);
//right forward
       motor(0,-50);
    motor(2,-50);
```

```
  }
  ao();
  msleep(500);
}

void grab_Bigcube(){                    //Grabs an object on the platform

    set_servo_position(2, dpince_open);
    set_servo_position(3, gpince_open);
    set_servo_position(1, arm_up);
    msleep(500);
    forward(3);
    set_servo_position(2, dpince_close);
    set_servo_position(3, gpince_close);

}

int main(){
  while(1){
       int lumiere = analog(3);
       if (lumiere <770){

             enable_servos();
             set_servo_position(1, arm_up);
             backward(10);
        forward_line();
        backward(40);
             left(17);
             backward(30);
             turn_left(1600,50);
             backward(3);
        right(4);
        forward(4);
             grab_Bigcube();
        msleep(1000);
        backward(3);
        turn_right(2000,50);
        forward(50);
        right(15);
        forward_line();

       }
  }
return 0;
}
```

C/C++

```
#include <stdio.h>
#include <kipr/wombat.h>

const int arm_up = 860;        //servo 1
const int arm_down = 1800;
const int arm_half = 1500;


const int dpince_open = 2000;  //servo 2
const int dpince_close = 1200;

const int gpince_open = 1100;  //servo 3
const int gpince_close = 1800;



void forward(int d) {                        //Avancer pour une
certaine distance en inch
    int target = (1870*d)/9.42;
    cmpc(0);
    while (gmpc(0) < target){
        motor(0,50);
        motor(3,50);
        motor(1,50);
        motor(2,50);
    }

    ao();
}

void turn_around() {                  // Tourner 180B0
    int target = 4200;                // adjust for your robot
    cmpc(0);

    while (gmpc(0) < target) {
```

```
        motor(0, 50);    // left side forward
        motor(2, 50);

        motor(1, -50);   // right side backward
        motor(3, -50);
    }

    ao();
}


void backward(int d) {                          //Avancer pour une
certaine distance en inch
    int target = -1*(1870*d)/9.42;
    cmpc(0);
    while (gmpc(0) > target){
        motor(0,-50);
        motor(3,-50);
        motor(1,-50);
        motor(2,-50);
    }

    ao();
}


void left(int d) {                          //gauche pour une
certaine distance en inch
    int target = (1870*d)/9.42;
    cmpc(0);
    while (gmpc(0) < target){
        motor(0,50);
        motor(1,-50);
        motor(2,-50);
        motor(3,50);
    }
```

```
    ao();
}

void right(int d) {                          //gauche pour une
certaine distance en inch
    int target = (1870*d)/9.42;
    cmpc(1);
    while (gmpc(1) < target){
        motor(0,-50);
        motor(1,50);
        motor(2,50);
        motor(3,-50);
    }

    ao();
}

void forward_line() {                        //Avancer jusqu'a une
ligne noire
    while (1) {
     int light_value_1 = analog(1);
     int light_value_2 = analog(2);

     if (light_value_1 >= 1200 && light_value_2 >= 1200) {
          break;
     }

     motor(0,50);
     motor(1,50);
     motor(2,50);
     motor(3,50);
     }
     ao();

}
```

```
void turn_left(int angle, int speed) {
    cmpc(2);  // reset left encoder

    while (gmpc(2) < angle) {

        // LEFT side backward
        motor(2, -speed);
        motor(3, -speed);

        // RIGHT side forward
        motor(0, speed);
        motor(1, speed);
    }

    ao();
    msleep(300);
}


void turn_right(int angle, int speed) {
    cmpc(2);  // reset left encoder

    while (gmpc(2) < angle) {

        // LEFT side backward
        motor(2, speed);
        motor(3, speed);

        // RIGHT side forward
        motor(0, -speed);
        motor(1, -speed);
    }

    ao();
    msleep(300);
```

```
}


void grab_Bigcube(){              //Grabs an object on the platform

    set_servo_position(2, dpince_open);
    set_servo_position(3, gpince_open);
    set_servo_position(1, arm_down);
    msleep(500);
    forward(3);
    set_servo_position(2, dpince_close);
    set_servo_position(3, gpince_close);

}

int main(){
  while(1){
      int lumiere = analog(3);
      if (lumiere <770){
            enable_servos();
            set_servo_position(1, arm_up);
            backward(5);
            left(3);
            backward(3);
            turn_left();
            left(12);
            forward(1);
            grab_Bigcube();




      }
  }
```

```
return 0;
}
```

C/C++

```
#include <stdio.h>
#include <kipr/wombat.h>


const int arm_up = 860;       //servo 1
const int arm_down = 1700;
const int arm_half = 1500;


const int dpince_open = 2000;  //servo 2
const int dpince_close = 1300;

const int gpince_open = 1100;  //servo 3
const int gpince_close = 2000;

void open_pince() {
    set_servo_position(2, dpince_open);
    set_servo_position(3, gpince_open);
}

void close_pince() {
    set_servo_position(2, dpince_close);
    set_servo_position(3, gpince_close);
}

/*void align() {
    int left_sensor, right_sensor;
    int alined = 0;   // alignC) = 0 veut dire que le robot n'est pas alignC)
    forward(0.5);
    while (alined == 0){
        left_sensor = analog(1);
        right_sensor = analog(2);


        motor(0,-20);
        motor(3,-20);
        motor(1,-20);
        motor(2,-20);
        }

        if (left_sensor < 1000) {
             while(1) {
             motor(0,-10);
                    motor(1,10);
```

```
                  motor(2,-10);
                  motor(3,10);
            if (right_sensor <1000) {
                  alined = 1;
                              break;
               }
           }
       }

       if (right_sensor < 1000) {
            while(1) {
            motor(0,10);
                  motor(1,-10);
                  motor(2,10);
                  motor(3,-10);
            if (left_sensor <1000) {
                  alined = 1;
                              break;
               }
            }
      }

    ao();
}
*/

void forward(int d) {                                  //Avancer pour une certaine
distance en inch
    int target = (1870*d)/9.42;
    cmpc(0);
    while (gmpc(0) < target){
        motor(0,50);
        motor(3,50);
        motor(1,50);
        motor(2,50);
    }

    ao();
}

void turn_around() {                 // Tourner 180B0
    int target = 3500;               // adjust for your robot
    cmpc(0);

    while (gmpc(0) < target) {
```

```
        motor(0, 80);    // left side forward
        motor(2, 80);

        motor(1, -80);   // right side backward
        motor(3, -80);
    }

    ao();
}


void turn_fraud() {                 // Tourner 180B0
    int target = -800;               // adjust for your robot
    cmpc(0);

    while (gmpc(0) < target) {
        motor(0, -50);    // left side forward
        motor(2, -50);

        motor(1, 50);   // right side backward
        motor(3, 50);
    }

    ao();
}

void gturn_half() {                 // Tourner 180B0
    int target = 2000;               // adjust for your robot
    cmpc(0);

    while (gmpc(0) < target) {
        motor(0, 50);    // left side forward
        motor(2, 50);

        motor(1, -50);   // right side backward
        motor(3, -50);
    }

    ao();
}

void dturn_half() {                 // Tourner 180B0
    int target = -2000;               // adjust for your robot
    cmpc(0);
```

```
    while (gmpc(0) > target) {
        motor(0, -50);    // left side forward
        motor(2, -50);

        motor(1, 50);   // right side backward
        motor(3, 50);
    }

    ao();
}


void backward(int d) {                              //Reculer pour une certaine
distance en inch
    int target = -1*(1870*d)/9.42;
    cmpc(0);
    while (gmpc(0) > target){
        motor(0,-50);
        motor(3,-50);
        motor(1,-50);
        motor(2,-50);
    }

    ao();
}

void left(int d) {                            //gauche pour une certaine distance
en inch
    int target = (1870*d)/9.42;
    cmpc(0);
    while (gmpc(0) < target){
        motor(0,50);
        motor(1,-50);
        motor(2,-50);
        motor(3,50);
    }

    ao();
}

void right(int d) {                           //gauche pour une certaine distance
en inch
    int target = (1870*d)/9.42;
    cmpc(1);
    while (gmpc(1) < target){
```

```
        motor(0,-50);
        motor(1,50);
        motor(2,50);
        motor(3,-50);
    }

    ao();
}

void forward_line() {                                   //Avancer jusqu'a une ligne
noire
    while (1) {
        int left_sensor, right_sensor;
        left_sensor = analog(1);
        right_sensor = analog(2);

        if (left_sensor >= 1200 && right_sensor >= 1200) {
            break;
        }

        motor(0,80);
        motor(1,80);
        motor(2,80);
        motor(3,80);
    }
    ao();

}

void grab_Bigcube(){                    //Grabs an object on the platform

    set_servo_position(2, dpince_open);
    set_servo_position(3, gpince_open);
    set_servo_position(1, arm_up);
    msleep(500);
    forward(7);
    set_servo_position(2, dpince_close);
    set_servo_position(3, gpince_close);
    msleep(1000);
    backward(6);

}

void grab_smallcube(){                  //Grabs an object on the platform
```

```
    open_pince();
    set_servo_position(1, arm_down);
    msleep(500);
    set_servo_position(2, dpince_close);
    set_servo_position(3, gpince_close);

}



int main() {
    while(1){

       int lumiere = analog(3);
f
        if (lumiere <770){

            enable_servos();
            clear_motor_position_counter(0);
            clear_motor_position_counter(1);
            clear_motor_position_counter(2);
            clear_motor_position_counter(3);
            set_servo_position(1, arm_up);



            forward(10);
            forward_line();

            left(3);
            grab_Bigcube();
            left(27);
            turn_around();
            right(6);
            left(1);


            forward(5);
            forward_line();
            forward(3);
            turn_fraud();

            msleep(1000);
             for (int i=0;i<1500;i+=10){
                    set_servo_position(1,860+i);
```

```
                    msleep(5);
                    }

            msleep(1000);
            set_servo_position(2, dpince_open);
            set_servo_position(3, gpince_open);
            msleep(1000);
            set_servo_position(1, arm_up);
            msleep(1000);


            backward(3);

        }
    }
    return 0;

}
```

```
#include <stdio.h>
#include <kipr/wombat.h>
#include <signal.h>
#include <time.h>

const int arm_up = 860;            //servo 1
const int arm_down = 1700;
const int arm_half = 1500;


const int dpince_open = 2000;  //servo 2
const int dpince_close = 1300;

const int gpince_open = 860;  //servo 3
const int gpince_close = 2000;
int end;
void open_pince() {
   set_servo_position(2, dpince_open);
   set_servo_position(3, gpince_open);
}

void close_pince() {
   set_servo_position(2, dpince_close);
   set_servo_position(3, gpince_close);
}

/*void align() {
   int left_sensor, right_sensor;
   int alined = 0;   // alignC) = 0 veut dire que le robot n'est pas alignC)
   forward(0.5);
   while (alined == 0){
      left_sensor = analog(1);
      right_sensor = analog(2);


      motor(0,-20);
      motor(3,-20);
      motor(1,-20);
      motor(2,-20);
      }

      if (left_sensor < 1000) {
         while(1) {
                  motor(0,-10);
```

```
                motor(1,10);
                motor(2,-10);
                motor(3,10);
                if (right_sensor <1000) {
            alined = 1;
                                        break;
          }
        }
      }

      if (right_sensor < 1000) {
        while(1) {
                motor(0,10);
                motor(1,-10);
                motor(2,10);
                motor(3,-10);
                if (left_sensor <1000) {
            alined = 1;
                                        break;
          }
        }
        }

  ao();
}
*/

void forward(int d) {                                   //Avancer pour une certaine distance en
inch
  int target = (1870*d)/9.42;
  cmpc(0);
  while (gmpc(0) < target){
    motor(0,50);
    motor(3,50);
    motor(1,50);
    motor(2,50);
  }

  ao();
}

void turn_around() {              // Tourner 180B0
  int target = 3500;              // adjust for your robot
  cmpc(0);
```

```
    while (gmpc(0) < target) {
        motor(0, 80);    // left side forward
        motor(2, 80);

        motor(1, -80);   // right side backward
        motor(3, -80);
    }

    ao();
}


void turn_fraud() {                 // Tourner 180B0
    int target = 400;               // adjust for your robot
    cmpc(0);

    while (gmpc(0) < target) {
        motor(0, 50);    // left side forward
        motor(2, 50);

        motor(1, -50);   // right side backward
        motor(3, -50);
    }

    ao();
}

void turn_fraud2() {                 // Tourner 180B0
    int target = -100;               // adjust for your robot
    cmpc(0);

    while (gmpc(0) > target) {
        motor(0, -50);
        motor(2, -50);

        motor(1, 50);
        motor(3, 50);
    }

    ao();
}

void let_go(){
```

```
    set_servo_position(1,1500);
      for (int i=0;i<dpince_open;i+=3){
         set_servo_position(2,dpince_close+i);
         set_servo_position(3,gpince_close-2i);
         msleep(5);
      }
      msleep(1000);
      set_servo_position(1, arm_up);
      msleep(1000);
}


void gturn_half() {                // Tourner 180B0
   int target = 2000;               // adjust for your robot
   cmpc(0);

   while (gmpc(0) < target) {
      motor(0, 50);    // left side forward
      motor(2, 50);

      motor(1, -50);   // right side backward
      motor(3, -50);
   }

   ao();
}

void dturn_half() {                // Tourner 180B0
   int target = -2000;               // adjust for your robot
   cmpc(0);

   while (gmpc(0) > target) {
      motor(0, -50);    // left side forward
      motor(2, -50);

      motor(1, 50);   // right side backward
      motor(3, 50);
   }

   ao();
}


void backward(int d) {                                  //Reculer pour une certaine distance en inch
```

```
    int target = -1*(1870*d)/9.42;
    cmpc(0);
    while (gmpc(0) > target){
        motor(0,-75);
        motor(3,-75);
        motor(1,-75);
        motor(2,-75);
    }

    ao();
}

void left(int d) {                                  //gauche pour une certaine distance en inch
    int target = (1870*d)/9.42;
    cmpc(0);
    while (gmpc(0) < target){
        motor(0,75);
        motor(1,-75);
        motor(2,-75);
        motor(3,75);
    }

    ao();
}

void right(int d) {                                 //gauche pour une certaine distance en inch
    int target = (1870*d)/9.42;
    cmpc(1);
    while (gmpc(1) < target){
        motor(0,-75);
        motor(1,75);
        motor(2,75);
        motor(3,-75);
    }

    ao();
}

void forward_line() {                               //Avancer jusqu'a une ligne noire
    while (1) {
        int left_sensor, right_sensor;
        left_sensor = analog(1);
        right_sensor = analog(2);
```

```
        if (left_sensor >= 1200 && right_sensor >= 1200) {
            break;
        }

        motor(0,90);
        motor(1,90);
        motor(2,90);
        motor(3,90);
    }
    ao();

}

void grab_Bigcube(){                    //Grabs an object on the platform


    set_servo_position(2, dpince_open);
    set_servo_position(3, gpince_open);
    set_servo_position(1, arm_up);
    msleep(500);
    forward(7);
    set_servo_position(2, dpince_close);
    set_servo_position(3, gpince_close);
    msleep(1000);
    backward(6);

}

void grab_smallcube(){                          //Grabs an object on the platform

    open_pince();
    set_servo_position(1, arm_down);
    msleep(500);
    set_servo_position(2, dpince_close);
    set_servo_position(3, gpince_close);

}

void turn_left(int angle, int speed){

  cmpc(2);
  while(gmpc(2) < angle){
          //left backward
    motor(1,-50);
```

```
        motor(3,-50);
//right forward
        motor(0,50);
   motor(2,50);
  }
  ao();
  msleep(500);
}

void turn_right(int angle, int speed){

  cmpc(2);
  while(gmpc(2) >- angle){
        //left backward
   motor(1,50);
        motor(3,50);
//right forward
        motor(0,-50);
   motor(2,-50);
  }
  ao();
  msleep(500);
}



int main() {
        end = 0;
   while(end == 0){
      int lumiere = analog(3);

      if (lumiere <770){
      shut_down_in(119);

      enable_servos();
      clear_motor_position_counter(0);
      clear_motor_position_counter(1);
      clear_motor_position_counter(2);
      clear_motor_position_counter(3);
      set_servo_position(1, arm_up);
```

```
forward(10);
forward_line();

left(3);
forward(4);
grab_Bigcube();
backward(4);
left(22);
turn_around();
right(5);
left(1.5);


forward(5);
forward_line();
forward(3);
turn_fraud();

msleep(1000);

set_servo_position(1,800);
msleep(1000);
for (int i=0;i<810;i+=5){
   set_servo_position(1,800+i);
   msleep(5);
}

set_servo_position(1,1500);
            let_go();
msleep(1000);
set_servo_position(1, arm_up);
msleep(1000);


backward(3);

set_servo_position(1, arm_up);
   //Fin depose 1er cube

            backward(5);
forward_line();
backward(40);
backward(20);
forward(20);
```

```
            left(17);
            backward(30);
            turn_left(1600,50);
            backward(3);
right(4);
forward(2.5);
            grab_Bigcube();
msleep(1000);

   //Attaper cube 2
backward(3);
turn_right(2000,50);
left(6);
forward_line();
forward(5);
forward_line();
forward(10);
right(15);
forward_line();
forward(3.5);
turn_fraud2();
set_servo_position(1,800);
msleep(1000);
for (int i=0;i<810;i+=5){
   set_servo_position(1,800+i);
   msleep(5);
}
            //Ouverture depose 2
let_go();
msleep(1000);
set_servo_position(1, arm_up);
msleep(1000);


set_servo_position(1, arm_up);
            backward(5);
forward_line();
backward(40);
            left(17);
            backward(30);
            turn_left(1600,50);
            backward(3);
right(4);
forward(4);
```

```
                grab_Bigcube();
        msleep(1000);
        backward(3);
        turn_right(2000,50);
        forward(50);
        right(15);
        forward_line();


        msleep(1000);

        set_servo_position(1,800);
        msleep(1000);
        for (int i=0;i<815;i+=5){
            set_servo_position(1,800+i);
            msleep(5);
        }

        let_go();
        msleep(1000);
        set_servo_position(1, arm_up);
        msleep(1000);
                backward(5);

                set_servo_position(1, arm_down);
        right(5);
        backward(60);
        msleep(1000);


        }
    }
    return 0;

}
```

Strategy is the key!

1. Analyze the various scoring opportunities.
2. Decide your scoring goals and multipliers.
3. Design the robots and accessories toward that goal.
4. Create the program to reach your goal.

We can design any part we want, including somthing to collect the small tubes (drums) that will be released starting 10s after start.

We cannot interfere with the other side of the board or the opponents' robots ⇒ **DSQ!**

3D prints must be grayscale!!!! (black, dark grey, grey or white)

Cannot use PVC foam or other materials cut or drilled.
No more than 6 pieces can be 3D printed.
We can use paper up to #20 - rubber band and wire but not metallic.

Don't forget to enable servos in the program, and disable them just before the end(return 0;)

Attention, le revêtement du sol de la compétition glisse beaucoup plus. Les déplacements latéraux (slide) ne sont pas aussi performants.

Using color detection is only useful if you aim for the cubes. Sorting the cubes worth a lot of points.
The poms are already sorted or close enough so no use of the camera.
The most interesting case of using the camera would be to detect the drums (small pipes) but that is a very challenging goal.

Maybe, we can think of designing a part to have them roll on our side.

You should look into driving using the gyrometer, helps with accuracy and straighter directions.

Small robot test program:

```C/C++
#include <stdio.h>
#include <kipr/wombat.h>
/*Table d'adressage
motor(0) : roue droite
motor(2) : roue gauche
reflex(a0) : sol droit
reflex(a1) : sol gauche
light(a3) : lumière
servo 1: front arm - low=2047 - high90=1000 - origin=500
*/
const int arm_up = 500;
const int arm_down = 2048;
const int arm_half = 1000;

int light_value_1;
int light_value_2;
int arm_pos=0;



int armLift(int pos){
    //Fonction pour contrôler les déplacements verticaux du bras de manière progressive
     if(pos>arm_pos){
        for (int i=arm_pos;i<=pos;i+=10){
           set_servo_position(1, i);
            msleep(10);
        }
    }
    else{
        for (int i=arm_pos;i>=pos;i-=10){
           set_servo_position(1, i);
            msleep(25);
        }
```

```
    }
    return pos;
}
// drive forward a distance in cm using motor 0's ticks
void forwardDist(int d_cm, int speed){
    int cible = (d_cm * 3600 / 20.4); // same conversion you used
    cmpc(2); // clear motor 0 counter
    while (gmpc(2) < cible){
        motor(0, -speed); // left motor on port 0
        motor(2, speed); // right motor on port 2
     }
    //Hard stop better than ao()
    motor(0, 0);
     motor(2, 0);
    msleep(250);
}

// turn right by angle degrees in place
void turnRight(int angle_deg, int speed ){
    int cible = angle_deg * 12.2; // 12 is an approximate ticks/deg
     cmpc(0);
     while (gmpc(0) < cible) {//Be careful of the encoder used and
the direction
        motor(0, speed); // left motor backward
        motor(2, speed); // right motor forward
    }
    //Hard stop better than ao()
    motor(0, 0);
     motor(2, 0);
    msleep(250);
}
void forwardLine() {//Avancer jusqu'a une ligne noire
    while (1) {
     light_value_1 = analog(0);
     light_value_2 = analog(1);
```

```
      if (light_value_1 > 3000 || light_value_2 > 3000) {
            //Hard stop better than a0()
            motor(0, 0);
            motor(2, 0);
            msleep(250);
           break;
      }
      motor(0,-30);
      motor(2,30);
      }
}
void squareUp(){
    light_value_1 = analog(0);
      light_value_2 = analog(1);
      while(light_value_1>=(light_value_2)){
         light_value_1 = analog(0);
      light_value_2 = analog(1);
         motor(2,15);
         motor(0,15);
    }
    while(light_value_1<=(light_value_2)){
         light_value_1 = analog(0);
      light_value_2 = analog(1);
         motor(0,-15);
         motor(2,-15);
    }
    //Hard stop better than a0()
    motor(0, 0);
      motor(2, 0);
    msleep(250);
}

int main(){
    enable_servos();
    //Initialize arm
    set_servo_position(1, arm_up);
```

```
    arm_pos=500;
    // go forward 90 cm
    //forwardDist(30, 100);
     forwardLine();
    squareUp();
    // turn right 90 degrees
    //turnRight(90, 40);
    //Arm down to push
     arm_pos=armLift(arm_down);
    msleep(250);
    // go forward 10 cm
    //forwardDist(10, 50);
    //arm_pos=armLift(arm_half);

    //Hard stop better than a0()
    motor(0, 0);
    motor(2, 0);
    msleep(250);
    disable_servos();
    return 0;
}
```

C/C++

```c
#include <stdio.h>
#include <kipr/wombat.h>
#include <signal.h>
#include <time.h>

const int arm_up = 900;
const int arm_down = 1813;
const int arm_half = 1300;

const int dpince_open = 0;
const int dpince_close = 560;

int end;
int left_sensor;
int right_sensor;

/* ---------------- SERVO FUNCTIONS ---------------- */
void open_pince() {
    set_servo_position(2, dpince_open);
}

void close_pince() {
    set_servo_position(2, dpince_close);
}

/* ---------------- MOVEMENT FUNCTIONS ---------------- */
void arreter(){//Arrêt freiner, plus rapide et coupe l'inertie
    motor(0,0);
    motor(1,0);
    motor(2,0);
    motor(3,0);
    msleep(50);//Pause pour garantir l'arrêt
}

void forward(float d) {//Avancer d'une distance en inch
    int target = (1870*d)/9.42;
    cmpc(0);
    while (gmpc(0) < target){
        motor(0,50);
        motor(1,50);
        motor(2,50);
        motor(3,50);
    }
    arreter();
```

```
}

void backward(float d) {//Reculer d'une distance en inch
    int target = -1*(1870*d)/9.42;
    cmpc(0);//Mettre codeur à 0
    while (gmpc(0) > target){
        motor(0,-75);
        motor(1,-75);
        motor(2,-75);
        motor(3,-75);
    }
    arreter();
}

void left(float d) {//Slide left distance en inch
    int target = (1870*d)/9.42;
    cmpc(0);
    while (gmpc(0) < target){
        motor(0,75);
        motor(1,-75);
        motor(2,-75);
        motor(3,75);
    }
    arreter();
}

void right(float d) {//Slide right distance en inch
    int target = (1870*d)/9.42;
    cmpc(1);
    while (gmpc(1) < target){
        motor(0,-40);
        motor(1,40);
        motor(2,40);
        motor(3,-40);
    }
    arreter();
}

void turn_around() {//Demi-tour
    int target = 3500;
    cmpc(0);
    while (gmpc(0) < target) {
        motor(0, 80);
        motor(2, 80);
        motor(1, -80);
```

```
        motor(3, -80);
    }
    arreter();
}

void turn_fraud() {
    int target = 500;
    cmpc(0);
    while (gmpc(0) < target) {
        motor(0, 50);
        motor(2, 50);
        motor(1, -50);
        motor(3, -50);
    }
    arreter();
}

void turn_fraud2() {
    int target = -175;
    cmpc(0);
    while (gmpc(0) > target) {
        motor(0, -50);
        motor(2, -50);
        motor(1, 50);
        motor(3, 50);
    }
    arreter();
}

void forward_line() {//Avancer jusqu'à une ligne noire
    while (1) {
       left_sensor = analog(0);
       right_sensor = analog(1);

        if (left_sensor >= 1200 || right_sensor >= 1200) {//Un seul des capteurs
peut suffire
            break;
        }

        motor(0,90);
        motor(1,90);
        motor(2,90);
        motor(3,90);
    }
    arreter();
```

```
}
void square_up(){
    while(1){
             left_sensor = analog(0);
             right_sensor = analog(1);

    // BOTH on black b aligned
    if (left_sensor > 200 && right_sensor  > 200){
        arreter();
        break;
    }

    // LEFT sees black, RIGHT sees white b turn right
    else if (left_sensor > 200 && right_sensor  <= 200){
        motor(0, 40);
        motor(2, 40);
        motor(1, -40);
        motor(3, -40);
    }

    // RIGHT sees black, LEFT sees white b turn left
    else if (left_sensor <= 200 && right_sensor  > 200){
        motor(0, -40);
        motor(2, -40);
        motor(1, 40);
        motor(3, 40);
    }

    // BOTH on white b move forward slowly
    else {
        motor(0, 30);
        motor(1, 30);
        motor(2, 30);
        motor(3, 30);
    }
    msleep(10);
       }
}

/* ---------------- ARM + PINCE ---------------- */

void let_go(){
    set_servo_position(1, 1100);
    msleep(500);
```

```
    for (int pos = dpince_close; pos >= dpince_open; pos -= 5){
        set_servo_position(2, pos);
        msleep(5);
    }

    msleep(500);
    set_servo_position(1, arm_up);
    msleep(500);
}

void grab_Bigcube(){
    open_pince();
    set_servo_position(1, arm_up);
    msleep(500);

    forward(7);

    close_pince();
    msleep(1000);

    backward(6);
}

void grab_smallcube(){
    open_pince();
    set_servo_position(1, arm_down);
    msleep(500);
    close_pince();
    msleep(1000);
}

void grab_on_ground(){
    open_pince();
    set_servo_position(1, arm_down);
    msleep(500);
    close_pince();
    msleep(1000);
}


void Let_go_top(){
    set_servo_position(1, arm_up);
    msleep(500);
    open_pince();
    msleep(500);
```

```
    close_pince();
    msleep(1000);
}
/* ---------------- MAIN ---------------- */

int main() {
    enable_servos();
    set_servo_position(1, arm_up);
    open_pince();
    msleep(500);
    end = 0;

    while(end == 0){
        int lumiere = analog(3);

        if (lumiere < 770){

            while (1){
                    forward_line();
                square_up();
                msleep(100);
                arreter();
                forward(2.0);
                forward_line();
                square_up();
                msleep(100);
                arreter();
                msleep(50);
                backward(1.0);
                arreter();
                right(30);
                break;
            }
            end = 1;
        }
    }
       disable_servos();
    return 0;
}
```

C/C++

```
#include <kipr/wombat.h>

int main()
{
    printf("Hello World\n");
    camera_open(); // opens and establishes communication with the camera
    while(digital(0) == 0){ // loops until the a button is pressed
        camera_update(); // retrieves current camera image
       if(get_object_count(0) > 0){ // does the camera see at least 1 red object?
            printf("I see red.\n");
             }
        else{
            if(get_object_count(1) > 0){ // does the camera see at least 1 yellow
object?
                printf("I see yellow.\n");
            }
            else{
                if(get_object_count(2) > 0){ // does the camera see at least 1
green object?
                    printf("I see green.\n");
                }
                else{
                    printf("Where is the color?\n");
                }
            }
        }
       }
       camera_close(); // disconnects from the camera
    printf("End\n");
    return 0;
}
```